Name: _____
Surname: _____
Date: _____

Debugging (15 points each)

Instructions

The functions below contain one or more errors. You have to find these errors, explain why they happen, and suggest a fix to them.

Questions

1. The function below is designed to detect palindromes. It uses an auxiliary function inside. When we run the code, it raises an error on line 10: "`NameError: name "lower_and_join" is not defined`". Identify and fix the issue(s).

```
1  def is_palindrome(input_str: str) -> bool:
2
3      def lower_and_join(input_str: str) -> str:
4          return input_str.lower().replace(" ", "")
5
6      input_str = lower_and_join(input_str)
7      return input_str == input_str[::-1]
8
9  sentence = "Was it a car or a cat I saw"
10 print(lower_and_join(sentence))  # Expected: wasitacaroracatisaw
11 print(is_palindrome(sentence))  # Expected: True
```

Answer:

The error comes from the scope of `lower_and_join`. This function is defined **inside** `is_palindrome`, so its name is local: it only exists while `is_palindrome` is running. When we call it from the global scope in line 10, Python cannot find that name and raises a `NameError`. *(+7 points)*

We fix it by moving the definition of `lower_and_join` out of `is_palindrome`, into the global scope (removing its indentation, and defining it before it is used). Then both the `print` in line 10 and the function `is_palindrome` can use it. Deleting line 10 is not a good fix, as the expected output includes its result. *(+8 points)*

(Optional, fixed code)

```
1  def lower_and_join(input_str: str) -> str:
2      return input_str.lower().replace(" ", "")
3
4  def is_palindrome(input_str: str) -> bool:
5      input_str = lower_and_join(input_str)
6      return input_str == input_str[::-1]
7
8  sentence = "Was it a car or a cat I saw"
9  print(lower_and_join(sentence))  # Expected: wasitacaroracatisaw
10 print(is_palindrome(sentence))  # Expected: True
```

2. The code below is intended to accumulate values in a list every time the function is called. However, it behaves unexpectedly. Identify and fix the issue(s).

```python
def accumulate(value: int, lst: list=[]) -> list:
    lst.append(value)
    return lst

print(accumulate(1))  # Output: [1], Expected: [1]
print(accumulate(2))  # Output: [1, 2], Expected: [2]
print(accumulate(3, lst=[4]))  # Output: [4, 3], Expected: [4, 3]
```

Answer:

The problem is the default value of the parameter, `lst: list=[]`. Python creates that default list **only once**, when the function is defined, and not every time it is called. Therefore, all the calls that do not provide `lst` use (and modify) the same list. This is why the second call returns `[1, 2]` instead of `[2]`. It happens because lists are mutable. *(+7 points)*

The standard fix is to use an immutable default value, `None`, and create a new list inside the function when needed: `if lst is None: lst = []`. This creates a fresh list in every call. The third call, which provides its own list, keeps working as before. *(+8 points)*

(Optional, fixed code)

```python
def accumulate(value: int, lst: list=None) -> list:
    if lst is None:
        lst = []
    lst.append(value)
    return lst
```

3. The code below is intended to count the frequency of each character in a string. However, it raises a "`KeyError: "h"`" on the last line. Identify and fix the issue(s).

```
1 def char_frequency(s: str) -> dict:
2     freq = {}
3     for char in s.lower():
4         freq[char] += 1
5     return freq
6
7 print(char_frequency("hello"))
8 # Expected: {"h": 1, "e": 1, "l": 2, "o": 1}
```

Answer:

The problem is line 4, which is the same as doing `freq[char] = freq[char] + 1`. If the `char` is not defined in the dictionary first, it fails (which is what currently happens with the first "h"). *(+7 points)*

We should include an if-clause inside the for loop: check if `char` is already a key in the dictionary. If it is, add 1 as we already do. If not, create the key and initialize it to 1. *(+8 points)*

(Optional, fixed code)

```
1 def char_frequency(s: str) -> dict:
2     freq = {}
3     for char in s.lower():
4         if char in freq:
5             freq[char] += 1
6         else:
7             freq[char] = 1
8     return freq
```

4. This function is intended to increment the first element of an array by 1, the second by 2, the third by 3, etc. However, it doesn't return the expected result. Identify the problem(s) and fix the code.

```
1 import numpy as np
2
3 def increment_array(arr: np.array):
4     i = 1
5     for element in arr:
6         element = element + i
7         i += 1
8
9 numbers = np.array([1, 0, 1])
10 numbers = increment_array(numbers)
11 print(numbers)
12 # Expected Output: [2, 2, 4]
13 # Actual Output: None
```

Answer:

There are two problems. First, the function has no `return` statement, so it returns `None`. This is why the output is `None` and not an array. *(+5 points)*

Second, even if we returned `arr`, the values would not change. In the loop `for element in arr`, the variable `element` is only a copy of each value, and the line `element = element + i` just changes that variable, not the array. *(+5 points)*

To fix it, we need to write the new values back into the array, so we loop over the indices with `for idx in range(len(arr))` and assign `arr[idx] = arr[idx] + idx + 1`. The counter `i` is no longer needed, as the index provides it. Then we return the array. *(+5 points)*

Comment: A loop is not even needed. We can use `return arr + np.arange(1, len(arr) + 1)`, which also leaves the input array unchanged.

(Optional, fixed code)

```python
import numpy as np

def increment_array(arr: np.array):
    for idx in range(len(arr)):
        arr[idx] = arr[idx] + idx + 1
    return arr
```

Pseudo-Code

Instructions

In the following exercises, you will be asked to write Python functions by hand. Please follow these guidelines:

- Any build-in function you need is in the cheat-sheet provided.
- Clearly indicate each level of indentation using a right arrow (⇒).
- Ensure your code works correctly for any possible example.
- If your code has errors, it will be evaluated based on the provided docstring, comments, and explanations.

Full points will be awarded for code that performs as expected under all conditions.

5. Approximating Pi with Leibniz (20 points)

Write a function `approximate_pi` to estimate π using the Leibniz formula. This function should take one integer `n`, representing how many terms of the series to use. The function returns the estimated value of π.

$$\pi \approx 4 \times \sum_{k=0}^{n} \frac{(-1)^k}{2k+1}$$

Answer:

The function receives the integer `n` and returns a float. The formula is a sum, so we need an accumulator: before the loop we initialize `total = 0`. *(+4 points)*

The sum goes from $k = 0$ to $k = n$, both included, so we use a loop `for k in range(n + 1)`. Using `range(n)` would miss the last term. *(+6 points)*

Inside the loop, we compute the term $\frac{(-1)^k}{2k+1}$ with `(-1) ** k / (2 * k + 1)`, and add it to the accumulator with `total += ...`. The power of -1 alternates the sign of the terms. *(+6 points)*

After the loop, we multiply the accumulated sum by 4 and return it. *(+4 points)*

(Optional, code)

```
def approximate_pi(n: int) -> float:
    total = 0
    for k in range(n + 1):
        total += (-1) ** k / (2 * k + 1)
    return 4 * total
```

6. Culinary Preferences in Spain (20 points)

During your class's culinary trip around Spain, you have a list of foods to be tasted in each city. However, some classmates have dietary restrictions or preferences and have listed foods they won't eat. Your task is to accommodate everyone's preferences as much as possible.
Create a function `foods_for_all()`. It takes two inputs:

- `foods_by_city` a dictionary with city names and their food lists.
- `food_restrictions` a list of tuples with students' names and the foods they avoid.

The function `foods_for_all()` returns a dictionary with each city's food options that all students can eat. *(15 points)*
If a city ends up with no options for everyone, it is not included in the output. *(3 points)*
Also, ensure the original inputs remain unchanged after running your function. *(2 points)*

```
# Sample input
# Remember: your function should work with any other possibility!
foods_by_city = {
  "Madrid": ["Squid", "Omelette", "Churros", "Soup"],
  "Valencia": ["Paella", "Horchata", "Fartons"],
  "Cadiz": ["Fried fish", "Potato salad", "Prawns"]}

food_restrictions = [
("Alice", ["Squid", "Paella", "Prawns"]),
("Bob", ["Churros", "Fartons"]),
("Charlie", ["Prawns"]),
("Diana", ["Fried fish", "Paella"]),
("Eve", ["Squid", "Fartons"]),
("Fred", ["Churros", "Omelette", "Fartons"]),
("Gwen", ["Fried fish", "Prawns"]),
("Hilda", ["Squid", "Potato salad"]),
]
```

Answer:

First, we collect every food that at least one student avoids. We initialize an empty set, `forbidden = set()`. Then we loop over `food_restrictions` with `for name, foods in food_restrictions`, and in each step we add that student's foods with `forbidden.update(foods)` (or by using the union). A set is a good choice, as it removes duplicates and makes checking membership fast. *(+5 points)*

Second, we loop over the cities, `for city, foods in foods_by_city.items()`. For each city we keep only the foods that are not in the forbidden set, using a list comprehension: `allowed = [f for f in foods if f not in forbidden]`. *(+7 points)*

Third, we store the result in a new dictionary that we initialize as empty before the loop: `result[city] = allowed`. *(+3 points)*

To leave out the cities without options, we place that assignment inside an if-clause that checks that `allowed` is not empty, `if len(allowed) > 0`. *(+3 points)*

Finally, the original inputs remain unchanged because we never modify them: the list comprehension creates a new list for every city, and the result is stored in a new dictionary. Using `remove()` on the original lists would break this. *(+2 points)*

With the sample input, the function returns `{"Madrid": ["Soup"], "Valencia": ["Horchata"]}`, and Cadiz is not included.

(Optional, code)

```python
def foods_for_all(foods_by_city: dict, food_restrictions: list) -> dict:
    # Every food that someone avoids
    forbidden = set()
    for name, foods in food_restrictions:
        forbidden.update(foods)

    result = {}
    for city, foods in foods_by_city.items():
        # New list: the original one is not modified
        allowed = [f for f in foods if f not in forbidden]
        # Cities with no options are left out
        if len(allowed) > 0:
            result[city] = allowed
    return result
```

Cheatsheet

Given a number named `x`:

- `x % y` returns the remainder of the division $\frac{x}{y}$.
- `x // y` returns the quotient of the division $\frac{x}{y}$.
- `abs(x)` returns the absolute value of `x`.

Given a string named `s`:

- `int(s)` returns an integer. Fails if `s` has invalid characters.
- `float(s)` returns a floating number. Fails if `s` has invalid characters.
- `s.lower()` returns a copy of the string, all in lowercase.
- `s.split(" ")` split a string into a list where each word is a list item.
- `s.count(<pattern>)` returns the number of non-overlapping occurrences of `<pattern>`.
- `s.find(<pattern>, <start>)` finds the first occurrence of the specified `<pattern>`, starting from index `<start>`. Returns `-1` if the pattern is not found.
- `s.replace(<pattern>, <new>)` returns a new string where all instances of `<pattern>` have been replaced with `<new>`.

Given a list named `ls`:

- `ls.append(element)` add element to that list.

Given a list or tuple named `ls`:

- `ls.index(element)` returns the index of element in that list.
- `len(ls)` returns the number of elements in that list.
- `sum(ls)` returns the sum of the elements of that list.
- `sorted(ls)` returns a sorted version of that list.
- `tuple(ls)` / `set(ls)` returns a tuple / set using the elements of that list.
- `", ".join(ls)` returns a string with the elements of the list separated by commas.

Given a set named `a`:

- `a.add(x)` adds the element `x` to the set `a`.
- `a.union(b)` or (`a | b`) returns the union of set `a` with another set `b`.
- `a.intersection(b)` or (`a & b`) returns the intersection of set `a` with another set `b`.
- `a.difference(b)` or (`a - b`) returns the difference of set `a` with another set `b`.

Given a dictionary named `d`:

- `list(d.keys())` returns a list with the keys of `d`.
- `list(d.values())` returns a list with the values of `d`.
- `list(d.items())` returns a list with the tuples (key, value) of `d`.

Some useful build-in functions:

- `type(x)` returns the type of `x` (int, float, list...).
- `range(start, stop)` generates numbers from `start` until `stop`, not including `stop`.
- `input(msg)` is used to take user input, in form of a string.

NumPy Cheatsheet

Importing NumPy:

- `import numpy as np`
- `rng = np.random.default_rng()`

Creating arrays:

- `np.zeros((m, n))` creates an $m \times n$ array filled with zeros.
- `np.ones((m, n))` creates an $m \times n$ array filled with ones.
- `np.arange(start, stop, step)` creates values from `start` to `stop` (exclusive) with a given `step`.
- `np.linspace(start, stop, num)` creates `num` evenly spaced values between `start` and `stop`.

Given a NumPy array `arr`:

- `arr.ndim` returns the number of array dimensions, as a number.
- `arr.shape` returns the shape of an array, as a tuple.

Array operations:

- `np.dot(a, b)` computes the dot product.
- `np.matmul(a, b)` or `a @ b` performs matrix multiplication.
- `np.transpose(a)` or `a.T` transposes the array.

Array manipulation:

- `a.reshape((m, n))` changes the shape of an array.
- `a.flatten()` flattens a multidimensional array into 1D.
- `np.concatenate((a, b), axis=0)` joins arrays along an axis.

- `np.stack(<sequence>)` joins a sequence of arrays along a new axis.
- `np.vstack((a, b))`, `np.hstack((a, b))` vertical/horizontal stacking.
- `np.split(a, n)` splits an array into `n` parts.

Statistics and descriptive analysis:

- `np.mean(a)` mean of all elements.
- `np.median(a)` median of all elements.
- `np.std(a)` standard deviation.
- `np.var(a)` variance.
- `np.min(a)`, `np.max(a)` minimum and maximum.
- `np.argmin(a)`, `np.argmax(a)` indices of minimum and maximum.
- `np.corrcoef(a, b)` correlation coefficient matrix.
- `np.cov(a, b)` covariance matrix.

Linear algebra:

- `np.linalg.inv(a)` inverse of matrix `a`.
- `np.linalg.det(a)` determinant of `a`.
- `np.linalg.eig(a)` eigenvalues and eigenvectors.
- `np.linalg.norm(a)` vector or matrix norm.
- `np.linalg.solve(a, b)` solves $Ax = b$.

Logical operations and conditions:

- `np.where(cond, x, y)` selects elements from `x` or `y` based on condition.
- `np.all(a)`, `np.any(a)` check if all/any elements are True.
- `np.unique(a)` returns sorted unique elements.

Random number generation (using `rng`):

- `rng.random(size)` uniform distribution in $[0, 1)$.
- `rng.integers(low, high, size)` random integers in $[low, high)$.
- `rng.normal(mean, std, size)` normal (Gaussian) distribution.
- `rng.uniform(low, high, size)` uniform distribution in $[low, high)$.
- `rng.choice(a, size)` random sample from array `a`.

Random number generation (legacy API):

- `np.random.randint(low, high, size)` returns random integers from low (inclusive) to high (exclusive).
- `np.random.uniform(low, high, size)` samples from a uniform distribution.
- `np.random.seed(seed)` fixes the seed. After a seed reset, the same set of numbers will appear every time.

Math and trigonometry:

- `np.power(arr1, arr2)` returns the first array elements raised to powers from second array, element-wise.
- `np.sin(arr)` returns the trigonometric sine, assuming angle in radians.
- `np.cos(arr)` returns the trigonometric cosine, assuming angle in radians.
- `np.radians(arr)` converts angles from degrees to radians.
- `np.degrees(arr)` converts angles from radians to degrees.